Dynamic Exposure Rebalancing Exercise

Problem Statement

You are tasked with redistributing "exposures" between entities in a portfolio to satisfy constraints while optimizing the portfolio. The goal is to ensure the portfolio adheres to specific constraints and rules after rebalancing.

Entities and Attributes

Each entity has:
- EntityId: A unique identifier.
- Exposure: Current exposure amount (positive decimal).
- Capacity: Maximum allowable exposure for this entity.
- Priority: A rank indicating the importance of the entity (lower values indicate higher priority).

Requirements

Implement a method Rebalance to redistribute exposures between entities. The solution must:

1. Satisfy Constraints:
   - No entity's exposure may exceed its Capacity.
   - The total exposure across all entities must remain unchanged.

2. Minimize Changes:
   - Aim to minimize the number of redistributions or splits.

3. Adhere to Rules:
   - Prioritize low-priority entities (higher Priority value) for receiving redistributed exposure.

4. Handle Special Cases:
   - If it is not possible to rebalance due to constraints, handle the situation gracefully and explain your approach.

Example Input

```
Entities:
Entity A: Exposure = 40, Capacity = 50, Priority = 1
Entity B: Exposure = 30, Capacity = 60, Priority = 2
Entity C: Exposure = 20, Capacity = 40, Priority = 3
Entity D: Exposure = 10, Capacity = 20, Priority = 4
```

Class Skeleton

```
public class Entity {
    public string EntityId { get; }
    public decimal Exposure { get; }
    public decimal Capacity { get; }
    public int Priority { get; }

    public Entity(string entityId, decimal exposure, decimal capacity, int priority) {
        EntityId = entityId;
        Exposure = exposure;
        Capacity = capacity;
        Priority = priority;
    }
}

public class ExposureBalancer {
    private List<Entity> Entities;
```

```
    public ExposureBalancer(List<Entity> entities) {
        Entities = entities;
    }

    public void Rebalance() {
        // TODO: Implement redistribution logic
    }

    public decimal GetTotalExposure() {
        return Entities.Sum(e => e.Exposure);
    }

    public bool IsValid() {
        return Entities.All(e => e.Exposure <= e.Capacity);
    }
}
```

Submission Guidelines

1. Zip your solution and email it, or
2. Push it to a private GitHub repository and share access with the user static-m.

Evaluation Criteria

1. Correctness: Does the solution satisfy constraints and rules?
2. Tradeoff Analysis: Are decisions reasonable and well-justified?
3. Code Quality: Is the code clean, maintainable, and extensible?
4. Graceful Failure: Is the handling of unsolvable scenarios logical and clear?